

# Chapter 34 - Disk and File I/O

The Intuition Engine has a single disk volume. Programs read it, write it, and list its contents through one small block of memory registers. BASIC wraps that block behind four keywords: LOAD, SAVE, BLOAD, and the direct-mode command DIR. Machine code on any CPU can drive the block directly.

## 34.1 The disk volume

There is one disk. Every filename in this chapter names a file on that disk. The reader does not see paths outside it: a name like "GAME.BAS" refers to a file at the root of the volume, and a name like "music/title.mod" refers to a file in the music subdirectory of the volume.

Two rules apply to every filename:

- No leading /. Absolute names are rejected.
- No .. anywhere in the name. Parent-directory escapes are rejected.

A name that violates either rule produces a path-traversal error. The disk gate refuses the operation before it touches the volume.

## 34.2 The File I/O register block

The block lives at 0xF2200 and spans 32 bytes. Every register is 32-bit unless the chapter says otherwise.

| Address | Name            | R/W | Meaning                                                                                          |
|---------|-----------------|-----|--------------------------------------------------------------------------------------------------|
| 0xF2200 | FILE_NAME_PTR   | W   | Pointer to a NUL-terminated filename string                                                      |
| 0xF2204 | FILE_DATA_PTR   | W   | Pointer to the data buffer (read target / write source / list target)                            |
| 0xF2208 | FILE_DATA_LEN   | W   | Buffer length in bytes (for write)                                                               |
| 0xF220C | FILE_CTRL       | W   | Operation: 1 = read, 2 = write, 3 = list. Writing this register fires the operation immediately. |
| 0xF2210 | FILE_STATUS     | R   | 0 = OK, 1 = error                                                                                |
| 0xF2214 | FILE_RESULT_LEN | R   | Bytes actually transferred (after read or list)                                                  |
| 0xF2218 | FILE_ERROR_CODE | R   | 0 = OK, 1 = not found, 2 = permission, 3 = path traversal                                        |

The operation enums also have symbolic names: FILE\_OP\_READ = 1, FILE\_OP\_WRITE = 2, FILE\_OP\_LIST = 3. The error enums are FILE\_ERR\_OK, FILE\_ERR\_NOT\_FOUND, FILE\_ERR\_PERMISSION, FILE\_ERR\_PATH\_TRAVERSAL.

The block is synchronous. The store to FILE\_CTRL does not return until the operation has finished and the result registers reflect its outcome. There is no busy bit and no interrupt.

## 34.3 The four operations

### 34.3.1 Read

Steps:

1. Place a NUL-terminated filename somewhere in memory. Write its address to FILE\_NAME\_PTR.

2. Write the address of a destination buffer to `FILE_DATA_PTR`.
3. Write 1 to `FILE_CTRL`.
4. Read `FILE_STATUS`. If it is 0, the file is in the buffer and `FILE_RESULT_LEN` holds its size in bytes. If it is 1, read `FILE_ERROR_CODE` for the cause.

The disk gate does not write past the end of the file, but it also does not check the buffer size. The caller is responsible for allocating enough room. Names longer than 255 characters are truncated.

### 34.3.2 Write

Steps:

1. Place the filename and write its address to `FILE_NAME_PTR`.
2. Write the address of the source bytes to `FILE_DATA_PTR`.
3. Write the byte count to `FILE_DATA_LEN`.
4. Write 2 to `FILE_CTRL`.
5. Read `FILE_STATUS`.

Writing creates the file if it does not exist and replaces it if it does. There is no append mode in the register block; BASIC provides one through SAVE semantics for program text only.

### 34.3.3 List

Steps:

1. Place a directory name and write its address to `FILE_NAME_PTR`. The empty string lists the root.
2. Write the address of a target buffer to `FILE_DATA_PTR`.
3. Write 3 to `FILE_CTRL`.
4. Read `FILE_STATUS`.

On success the buffer holds a sorted list of entries, separated by CR LF, with a trailing CR LF after the last entry and a final NUL byte. Directory entries are listed with a trailing / to distinguish them from regular files. `FILE_RESULT_LEN` holds the length of the text in bytes (not counting the final NUL).

## 34.4 BASIC verbs

### 34.4.1 LOAD "name"

LOAD reads a BASIC program file from disk. Internally it calls the read operation against a built-in name buffer and data buffer. On success the program lines are tokenized and become the current program. The variable table is cleared. On a not-found result the machine prints `FILE NOT FOUND` and returns to the prompt with the previous program intact.

### 34.4.2 SAVE "name"

SAVE writes the current program to disk, detokenized into ASCII line-numbered text. The format round-trips through LOAD. There is no compression and no header.

### 34.4.3 BLOAD "name", addr

BLOAD reads a binary file straight into memory at `addr` and ignores any tokenization. Use it to load images, samples, fonts, and machine-code payloads. The trailing argument is the destination address. The file's contents land verbatim, byte by byte, starting at that address.

### 34.4.4 DIR

DIR is a direct-mode command. It calls the list operation against the root of the volume and prints the result. It has no tokenizer entry and no line-number form.

## 34.5 A complete read example

This fragment reads "FONT.RAW" into the buffer at 0x40000 and prints its size:

```
la      r1, name_ptr_reg
la      r2, font_name
store.l r2, (r1)

la      r1, data_ptr_reg
li      r2, #0x40000
store.l r2, (r1)

la      r1, ctrl_reg
li      r2, #1
store.l r2, (r1)

la      r1, status_reg
load.l  r3, (r1)
bnez    r3, .read_err

la      r1, result_len_reg
load.l  r4, (r1)
; R4 = bytes read
```

The constants name\_ptr\_reg, data\_ptr\_reg, ctrl\_reg, status\_reg, and result\_len\_reg resolve to 0xF2200, 0xF2204, 0xF220C, 0xF2210, and 0xF2214 respectively. font\_name points at the string "FONT.RAW", 0.

## 34.6 Error handling

A failed operation sets FILE\_STATUS to 1 and FILE\_ERROR\_CODE to one of:

| Code | Meaning                                                                                          |
|------|--------------------------------------------------------------------------------------------------|
| 1    | The file or directory does not exist on the volume.                                              |
| 2    | The volume refused the operation (read-only entry, locked path, or other permission constraint). |
| 3    | The filename used an absolute path or contained . . .                                            |

FILE\_RESULT\_LEN is zero after a failed list. After a failed read or write the result-length register is left undefined; do not rely on it.

## 34.7 Use from the small CPUs

The 6502 and Z80 reach the File I/O block through the same bus mapping that gives them the rest of the 0xFxxx region. Each register is 32-bit on the bus; the 8-bit CPUs write it as four bytes at the four consecutive byte offsets. The block also accepts byte-width writes to FILE\_CTRL: a write of 1, 2, or 3 to the low byte of FILE\_CTRL fires the matching operation. This is how the 8-bit CPUs trigger a read or a write without first assembling a 32-bit value.

The 16-bit address that maps to `FILE_CTRL` on the 6502 and the 8-bit Z80 bus is not in any bank-register window. The CPU adapter chapters (Chapters 26 and 27) describe the windowing scheme that exposes the wider `0xF2200` region to the smaller address spaces.